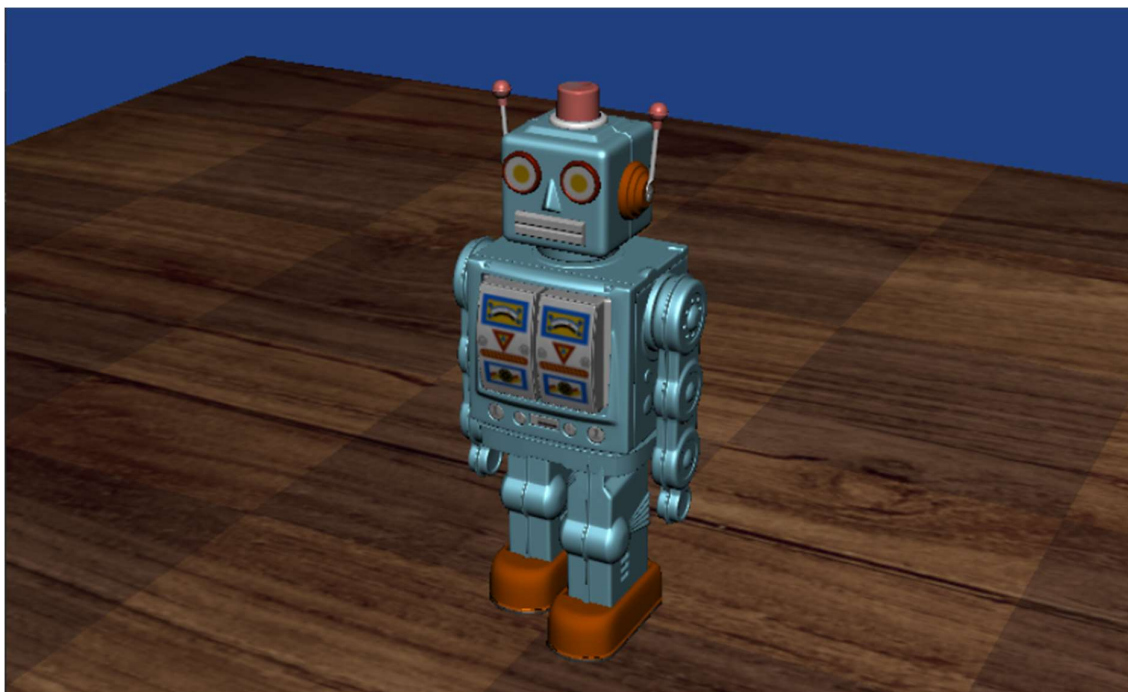


Számítógépes grafika minta ZH – Robot

Megoldás



0. Alapok

0.1. Kamera beállítása

Feladat

A kamera nézze a (0,0,0) pontot, és helyezkedjen el a (0,60,150) pontban. Világ felfele a (0,1,0).

Megoldás

A kamera paramétereit a **Camera** osztály **SetView** eljárásával módosíthatjuk. A függvény **3db glm : : vec3**-at vár az alábbi sorrendben:

1. A kamera pozíciója
2. A pont, melyre néz a kamera
3. A világtérben felfelé való irány

*FONTOS: A **CameraManipulator**, ami az egér/billentyű inputra forgatja/mozgatja a kamerát, saját belső állapotában tárol egy-egy snapshot-ot a fenti három paraméterről, s ebből számolja ki az új paramétereket, amikkel felülírja a kamera paramétereit. Ezért a kamera objektumunk minden egyes módosítása után meg kell hívni a **SetStateFromCamera** metódust, ami új snapshot-ot készít.*

A kiinduló projektben adottak az **m_camera** és az **m_cameraManipulator** adattagok, a kezdőállapotot az **Init** metódusban kell beállítani.

1. Geometriák

1.1. Asztal – Kocka

Feladat

A kiinduló projektben adott egy egység méretű, origó középpontú kocka geometria (a rajzoláshoz szükséges azonosítók az **m_cubeGPU** változóban érhetők el). Az objektum rajzolásához használjuk a **Wood_Table_Texture.png** textúrát.

Az asztal álljon 5 db megnyújtott kockából: az asztallap 200x10x200 egység méretű, az asztallábak 20x120x20 egység méretűek. Az asztallap úgy legyen elhelyezve, hogy a felső lapjának közepe éppen érintse a (0,0,0) pontot.

A négy láb szimmetrikusan helyezkedjen el, a felső lapjuk **essen egybe** az asztallap alsó lapjával. Az egyik láb legyen az $x = 60$, $z = 60$ pontba elhelyezve. Y érték a kapcsolódásnak megfelelően.

Megoldás

Az asztal megjelenítéséhez hozzunk létre külön eljárást **RenderTable** néven. A **RenderTable**-t a **Render**-ben hívjuk meg. Mozgassuk át a **glUseProgram** után a **DrawObject** hívásig bezárólag az összes utasítást a **RenderTable**-be és helyettük hívjuk meg a **RenderTable**-t.

A **RenderTable**-ben állítsuk be a **textúrát**, a **sampler**-t, majd a **DrawObject**-tel jelenítsük meg az asztallapot. A kockát már megkaptuk az **m_cubeGPU** adattagban, a world mátrix kiszámításához pedig rendelkezésünkre állnak a **ZHUtils.h** header-ben a **TABLE_SIZE** és a **TABLE_POS** konstansok.

Az asztallaphoz hasonlóan, ha azt szeretnénk, hogy az asztalláb felső lapja az $y=0$ síkon legyen, el kell tolnunk az asztallábat a magasságának felével. Ahhoz, hogy az asztallap alsó lapjával essen egybe, még az asztallap magasságával is el kell tolnunk. Az asztal középpontja az $x=0$ és $z=0$ pontban van, az erre való tükrözéssel az asztallábak középpontjai az $x=\pm 60$, $z=\pm 60$ pontba fognak esni. Itt szintén rendelkezésre áll a **TABLE_LEG_SIZE** konstans.

```
void CMyApp::RenderTable()
{
    //Textúraegységek beállítása
    glUniform1i(glGetUniformLocation("textureImage"), 0);

    //Textúrák beállítása, minden egységre külön
    glBindTextureUnit(0, m_woodTextureID);
    glBindSampler(0, m_SamplerID);

    glm::mat4 tableTransform = glm::translate(TABLE_POS);

    //Rajzolási parancs kiadása
    DrawObject(m_cubeGPU, tableTransform * glm::scale(TABLE_SIZE));

    for (int i = -1; i <= 1; i += 2) {
        for (int j = -1; j <= 1; j += 2) {
            glm::vec3 legPos = glm::vec3(i * 60, -(TABLE_LEG_SIZE.y +
                TABLE_SIZE.y) / 2.f, j * 60);
            DrawObject(m_cubeGPU, tableTransform * glm::translate(legPos) *
                glm::scale(TABLE_LEG_SIZE));
        }
    }
}
```



1.2. Robot

Feladat

A robot testrészeit töltsük be a megadott obj fájllokból, mindre közösen a Robot_Texture.png textúrát alkalmazzuk. Építsd fel a transzformációkat hierarchikusan a táblázatban megadottak alapján (lásd ábra is). A megadott vektorok a “szülő” objektum és a kérdéses al-objektum közötti relatív eltolást mutatják. Az itt leírtak helyett használható a whole_robot.obj, ekkor azonban erre a részre nem jár pont.

A következő testrészek legyenek elforgathatóak GUI-s Sliderek segítségével (ImGui::SliderAngle): a karok minden része külön-külön (az objektumok saját origóján átmenő X-tengely körül); a fej (először a saját X, aztán Y-tengely körül – két független szöggel). A Sliderek minimuma és maximuma úgy legyenek beállítva, hogy reális elfordulásokat kapjunk minden beállításra (azaz pl. a fej ne tudjon nagyon le- vagy felnézni, ne tudjon teljesen hátra fordulni).

A lábak legyenek animálva a megadott CalculateLegAnim(float time, float animOffset) függvény felhasználásával. Ez a függvény a táblázatban jelölthöz képest egy további eltolást határoz meg a lábakra. A time paraméter a program kezdete óta eltelt idő másodpercben (későbbi feladatban ezt változtatni kell majd!), míg az animOffset paraméter értéke az egyik lábra 0, a másik lábra π .

Megoldás

Az asztal textúrájához hasonlóan vegyünk fel a robot-nak is egy textúra azonosító adattagot mondjuk `m_robotTextureID` néven, majd töltsük be az `InitTextures` eljárásban, illetve ne felejtsük el kiegészíteni a `CleanTextures`-t sem!

A robothoz tartozó összes `file_name.obj`-hoz vegyünk fel egy-egy `m_robotFileNameGPU` `OGLObject` adattagot.

```
OGLObject m_robotHeadGPU{}, m_robotLeftArmLowerGPU{},  
m_robotLeftArmUpperGPU{}, m_robotLegGPU{}, m_robotRightArmLowerGPU{},  
m_robotRightArmUpperGPU{}, m_robotTorsoGPU{};
```

Töltsük be az `OGLObject`-eket az `InitGeometry` eljárásban az `m_cubeGPU`-hoz hasonlóan, az `ObjParser::parse` függvény segítségével. A fájlhoz vezető útvonalat a `main.cpp`-hez relatívan kell megadni. Ne felejtsük el a `CleanGeometry`-t is kiegészíteni!

```
m_headGPU = CreateGLObjectFromMesh(ObjParser::parse("Assets/head.obj"),  
    vertexAttribList);  
m_leftLowerArmGPU =  
    CreateGLObjectFromMesh(ObjParser::parse("Assets/left_lower_arm.obj"),  
        vertexAttribList);  
m_leftUpperArmGPU =  
    CreateGLObjectFromMesh(ObjParser::parse("Assets/left_upper_arm.obj"),  
        vertexAttribList);  
m_legGPU = CreateGLObjectFromMesh(ObjParser::parse("Assets/leg.obj"),  
    vertexAttribList);  
m_rightLowerArmGPU =  
    CreateGLObjectFromMesh(ObjParser::parse("Assets/right_lower_arm.obj"),  
        vertexAttribList);  
m_rightUpperArmGPU =  
    CreateGLObjectFromMesh(ObjParser::parse("Assets/right_upper_arm.obj"),  
        vertexAttribList);  
m_torsoGPU = CreateGLObjectFromMesh(ObjParser::parse("Assets/torso.obj"),  
    vertexAttribList);
```

Mivel itt az egyes transzformációk bonyolultabb számítást igényelnek, elkülönítjük őket és az `Update` eljárásban fogjuk előállítani a mátrixokat. Ehhez viszont először vegyük fel őket a header-ben:

```
glm::mat4 m_robotTransform, m_robotBaseTransform, m_robotHeadTransform,  
m_robotLeftArmUpperTransform, m_robotLeftArmLowerTransform,  
m_robotRightArmUpperTransform, m_robotRightArmLowerTransform,  
m_robotLeftLegTransform, m_robotRightLegTransform;
```

Az animációkhoz hozzunk létre új adatstruktúrát mondjuk `RobotState` névvel. Tartalmazzon egyelőre csak egy `float legTime` adattagot. Vegyünk fel egy ilyen típusú, `m_robotState` adattagot.

```
struct RobotState
{
    float legTime = 0.0f;
};
```

Vegyünk fel az ImGui paramétereket is mondjuk `m_robotLeftArmUpperRot`, `m_robotRightArmLowerRot`, `m_robotLeftArmLowerRot`, `m_robotRightArmLowerRot`, `m_robotHeadUpDownRot`, `m_robotHeadLeftRightRot` nevekkel. Egészítsük ki a `RenderGUI` eljárást egy ablak létrehozásával, ami az `ImGui::Begin` függvényhívással tehető meg. Egy filestream-hez hasonlóan az ablakot is be kell zárni az `ImGui::End` eljárás meghívásával. A `Begin` függvény egy `bool` értékkel tér vissza, ami azt reprezentálja, hogy rajzolásra került-e az ablak. Csak ebben az esetben kell kirajzolnunk a Slider-eket. Célszerű a kontroll csoportokat egy-egy `SeparatorText`-tel elválasztani. A forgatások korlátai legyenek mondjuk az alábbiak:

- Felkarok $\rightarrow -90^\circ - 180^\circ$
- Alkarok $\rightarrow -120^\circ - 120^\circ$
- Fej függőleges $\rightarrow -20^\circ - 50^\circ$
- Fej vízszintes $\rightarrow -120^\circ - 120^\circ$

```
if (ImGui::Begin("Beállítások"))
{
    ImGui::SeparatorText("Testrész-forgatások");
    ImGui::SliderAngle("Fej függőleges", &m_robotHeadUpDownRot, -20.0f, 50.0f);
    ImGui::SliderAngle("Fej vízszintes", &m_robotHeadLeftRightRot, -120.0f,
        120.0f);
    ImGui::SliderAngle("Bal alkar", &m_robotLeftArmLowerRot, -120.0f, 120.0f);
    ImGui::SliderAngle("Jobb alkar", &m_robotRightArmLowerRot, -120.0f,
        120.0f);
    ImGui::SliderAngle("Bal felkar", &m_robotLeftArmUpperRot, -90.0f, 180.0f);
    ImGui::SliderAngle("Jobb felkar", &m_robotRightArmLowerRot, -90.0f,
        180.0f);
}
ImGui::End();
```

Ezekkel rendelkezésünkre áll az összes szükséges adat. Térjünk át tehát az `Update` eljárásához.

A mátrixok kiszámítása előtt frissítsük a `legTime` tartalmát az előző frame óta eltelt idő hozzáadásával.

```
m_robotState.legTime += deltaTime;
```

A mátrixok előállításához most is rendelkezésünkre állnak a `ZHUtils.h` header-ben az alábbi konstansok: `RELATIVE_LEFT_ARM_LOWER`, `RELATIVE_RIGHT_ARM_LOWER`, `TORSO_POS`, `RELATIVE_LEFT_ARM_UPPER`, `RELATIVE_RIGHT_ARM_UPPER`, `RELATIVE_HEAD_POS`, `RELATIVE_LEFT_LEG_POS`, `RELATIVE_RIGHT_LEG_POS`, `AVG_EYE_POS_OFFSET`. Mivel a transzformációk hierarchikusan vannak megadva, az adat magával hordoz egy sorrendet is. A redundáns számítások elkerülése végett a hierarchikus fa gyökeréből indulunk ki, majd szint-folytonosan haladva fel tudjuk használni a korábbi számítások eredményeit:

```
m_robotBaseTransform =
    glm::translate(TORSO_POS);
m_robotHeadTransform =
    m_robotBaseTransform *
    glm::translate(RELATIVE_HEAD_POS) *
    glm::rotate(m_robotHeadLeftRightRot, glm::vec3(0, -1, 0)) *
    glm::rotate(m_robotHeadUpDownRot, glm::vec3(1, 0, 0));
m_robotLeftLegTransform =
    m_robotBaseTransform *
    glm::translate(RELATIVE_LEFT_LEG_POS) *
    glm::translate(CalculateLegAnim(m_robotState.legTime, 0.0f));
m_robotRightLegTransform =
    m_robotBaseTransform *
    glm::translate(RELATIVE_RIGHT_LEG_POS) *
    glm::translate(CalculateLegAnim(m_robotState.legTime, glm::pi<float>()));
m_robotLeftArmUpperTransform =
    m_robotBaseTransform *
    glm::translate(RELATIVE_LEFT_ARM_UPPER) *
    glm::rotate(m_robotLeftArmUpperRot, glm::vec3(-1, 0, 0));
m_robotRightArmUpperTransform =
    m_robotBaseTransform *
    glm::translate(RELATIVE_RIGHT_ARM_UPPER) *
    glm::rotate(m_robotRightArmUpperRot, glm::vec3(-1, 0, 0));
m_robotLeftArmLowerTransform =
    m_robotLeftArmUpperTransform * // !!
    glm::translate(RELATIVE_LEFT_ARM_LOWER) *
    glm::rotate(m_robotLeftArmLowerRot, glm::vec3(-1, 0, 0));
m_robotRightArmLowerTransform =
    m_robotRightArmUpperTransform * // !!
    glm::translate(RELATIVE_RIGHT_ARM_LOWER) *
    glm::rotate(m_robotRightArmLowerRot, glm::vec3(-1, 0, 0));
```

Hozzunk a robot kirajzolásához is létre új eljárást `RenderRobot` névvel. Az asztalhoz hasonlóan a textúraegység beállítása után a `DrawObject` meghívásával fogjuk kirajzolni a robot elemeit.

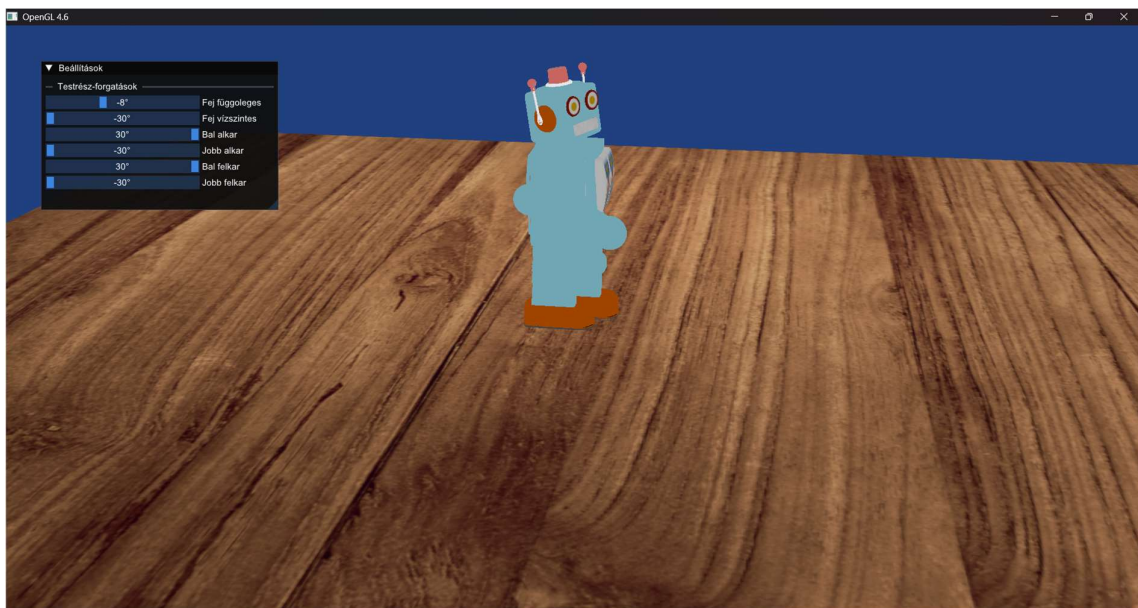
```

void CMyApp::RenderRobot()
{
    //Textúra
    glBindTextureUnit(0, m_robotTextureID);
    glBindSampler(0, m_SamplerID);
    glUniform1i( glGetUniformLocation("textureImage"), 0);

    DrawObject( m_robotTorsoGPU, m_robotBaseTransform );

    DrawObject( m_robotHeadGPU, m_robotHeadTransform );
    DrawObject( m_robotLegGPU, m_robotLeftLegTransform );
    DrawObject( m_robotLegGPU, m_robotRightLegTransform );
    DrawObject( m_robotLeftArmUpperGPU, m_robotLeftArmUpperTransform );
    DrawObject( m_robotRightArmUpperGPU, m_robotRightArmUpperTransform );
    DrawObject( m_robotLeftArmLowerGPU, m_robotLeftArmLowerTransform );
    DrawObject( m_robotRightArmLowerGPU, m_robotRightArmLowerTransform );
}

```



2. Shader feladatok

2.1. Alap árnyalás

Feladat

Minden megjelenő objektum legyen megvilágítva a Phong-modell szerint a következő paraméterekkel: irány fényforrás, ami felülről és kicsit oldalról,

$$\frac{1}{3}(1,2,2)$$

irányból érkezik.

Megoldás

Vegyünk fel adattagokat a fényforrásnak és az anyagjellemzőknek. Legyenek az alapértelmezett értékek ugyanazok, mint a gyakorlaton. A fény pozíciójának vegyük fel a kapott irányt.

Mivel a fényforrásunk irányfényforrás, a negyedik komponens 0.

```
glm::vec4 m_lightPosition = glm::vec4{1.0f, 2.0f, 2.0f, 0.0f} / 3.0f;
glm::vec3 m_La{0.0f, 0.0f, 0.0f}, m_Ld{1.0f, 1.0f, 1.0f}, m_Ls{1.0f, 1.0f, 1.0f};
GLfloat lightConstantAttenuation = 1.0f, m_lightLinearAttenuation = 0.0f,
      m_lightQuadraticAttenuation = 0.0f;

glm::vec3 m_Ka{1.0f, 1.0f, 1.0f}, m_Kd{1.0f, 1.0f, 1.0f}, m_Ks{1.0f, 1.0f, 1.0f};
GLfloat m_Shininess = 20.0f;
```

Bővítsük ki a **SetCommonUniforms** eljárást a fényforrás adattagonkénti átadásával, valamint adjuk át a kamera pozícióját is, ami a **Camera** példány **GetEye** függvényével kérhető le.

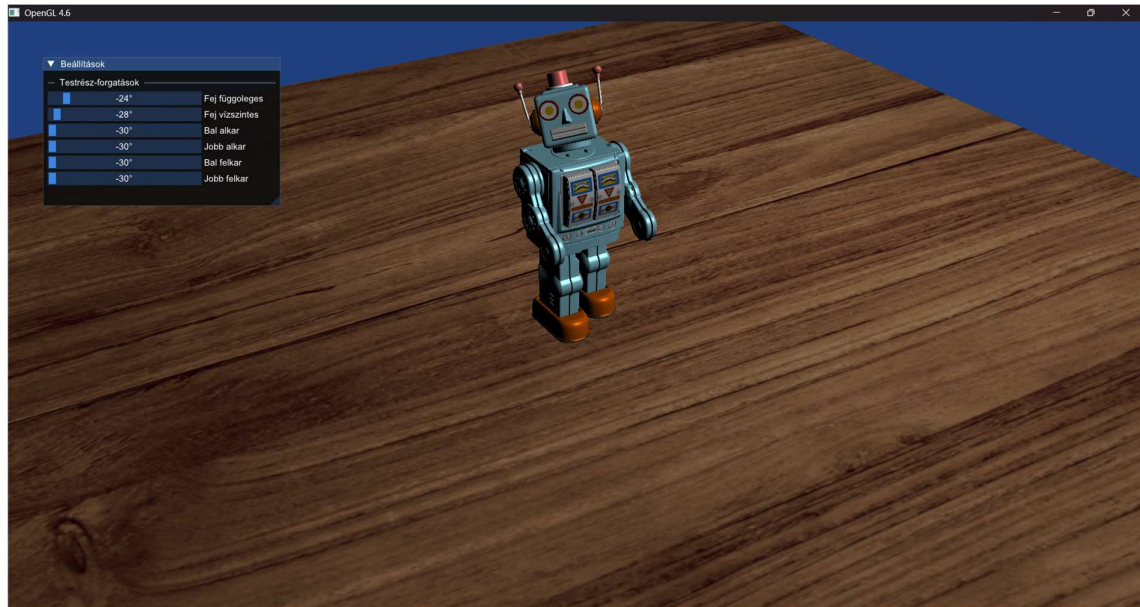
```
glUniform3fv( ul("cameraPosition"), 1, glm::value_ptr(m_camera.GetEye()));
glUniform4fv( ul("lightPosition"), 1, glm::value_ptr(m_lightProperties.pos));

glUniform3fv(ul("La"), 1, glm::value_ptr(m_lightProperties.La));
glUniform3fv(ul("Ld"), 1, glm::value_ptr(m_lightProperties.Ld));
glUniform3fv(ul("Ls"), 1, glm::value_ptr(m_lightProperties.Ls));

glUniform1f(ul("lightConstantAttenuation"),
      m_lightProperties.lightConstantAttenuation);
glUniform1f(ul("lightLinearAttenuation"),
      m_lightProperties.lightLinearAttenuation);
glUniform1f(ul("lightQuadraticAttenuation"),
      m_lightProperties.lightQuadraticAttenuation);
```

Az anyagjellemzőket ugyanígy, de külön-külön állítsuk be a **RenderRobot** és a **RenderTable** eljárásokban.

Induljunk ki ugyan abból a fragment shader-ből, mint ami a Lighting gyakorlat végén állt elő.



2.2. Asztal minta

Feladat

Az asztallapon legyen sakktábla minta a textúrákoordináták alapján a következők szerint:

$$val = \text{floor}(8 \cdot u) + \text{floor}(8 \cdot v)$$

Ha *val* értéke páratlan, akkor a textúrából kapott színt koordinátánként szorozzuk meg 0.8-del, különben maradjon az eredeti szín.

Ez a változtatás csak az asztallapon jelenjen meg, más objektumon ne (az asztal lábain sem).

Megoldás

Vegyünk fel három konstans egész szám adattagot, amelyek az éppen rajzolt, megkülönböztetendő objektumot jelölik:

```
const int SHADER_STATE_DEFAULT = 0;
const int SHADER_STATE_ROBOT   = 1;
const int SHADER_STATE_TABLE   = 2;
```

Bővítsük ki a RenderTable és a RenderRobot eljárásokat a uniform átadásával!

```
void CMyApp::RenderTable()
{
    //Anyagjellemzők beállítása
    glUniform3fv( ul("Ka"), 1, glm::value_ptr(m_Ka));
    glUniform3fv( ul("Kd"), 1, glm::value_ptr(m_Kd));
    glUniform3fv( ul("Ks"), 1, glm::value_ptr(m_Ks));

    glUniform1f( ul("Shininess"), m_Shininess);

    //Textúraegységek beállítása
    glUniform1i(ul("textureImage"), 0);

    //Textúrák beállítása, minden egységre külön
    glBindTextureUnit(0, m_woodTextureID);
    glBindSampler(0, m_SamplerID);

    glUniform1i( ul("state"), SHADER_STATE_TABLE);
    glm::mat4 tableTransform = glm::translate(TABLE_POS);

    //Rajzolási parancs kiadása
    DrawObject(m_cubeGPU, tableTransform * glm::scale(TABLE_SIZE));

    glUniform1i( ul("state"), SHADER_STATE_DEFAULT);
    for (int i = -1; i <= 1; i += 2) {
        for (int j = -1; j <= 1; j += 2) {
            glm::vec3 legPos = glm::vec3(i * 60, -(TABLE_LEG_SIZE.y +
                TABLE_SIZE.y) / 2.f, j * 60);
            DrawObject(m_cubeGPU, tableTransform * glm::translate(legPos) *
                glm::scale(TABLE_LEG_SIZE));
        }
    }
}
```

```

void CMyApp::RenderRobot()
{
    //Anyagjellemzők beállítása
    glUniform3fv( ul("Ka"), 1, glm::value_ptr(glm::vec3(1, 1, 1)));
    glUniform3fv( ul("Kd"), 1, glm::value_ptr(glm::vec3(1, 1, 1)));
    glUniform3fv( ul("Ks"), 1, glm::value_ptr(glm::vec3(1, 1, 1)));

    //Textúra
    glBindTextureUnit(0, m_robotTextureID);
    glBindSampler(0, m_SamplerID);
    glUniform1i( ul("textureImage"), 0);

    glUniform1i( ul("state"), SHADER_STATE_ROBOT);
    DrawObject( m_robotTorsoGPU, m_robotBaseTransform );

    DrawObject( m_robotHeadGPU, m_robotHeadTransform );
    DrawObject( m_robotLegGPU, m_robotLeftLegTransform );
    DrawObject( m_robotLegGPU, m_robotRightLegTransform );
    DrawObject( m_robotLeftArmUpperGPU, m_robotLeftArmUpperTransform );
    DrawObject( m_robotRightArmUpperGPU, m_robotRightArmUpperTransform );
    DrawObject( m_robotLeftArmLowerGPU, m_robotLeftArmLowerTransform );
    DrawObject( m_robotRightArmLowerGPU, m_robotRightArmLowerTransform );
}

```

Bővítsük ki a shader-t a sakktábla-árnyalással!

```

//Ugyan annak kell lennie, mint a MyApp.h-ban!
const int SHADER_STATE_DEFAULT = 0;
const int SHADER_STATE_ROBOT = 1;
const int SHADER_STATE_TABLE = 2;

uniform int state = SHADER_STATE_DEFAULT;

...

void main()
{
    ...

    vec4 texColor = texture(textureImage, textureCoords);
    if ( state == SHADER_STATE_TABLE )
    {
        uint val = uint(floor(8*textureCoords.x)+floor(8*textureCoords.y));
        vec3 mult = val % 2 == 0 ? lightColorMultiplier : darkColorMultiplier;
        texColor *= vec4(mult,1);
    }

    outputColor = vec4(shadedColor, 1) * texColor;

    // normal vector debug:
    // outputColor = vec4( normal * 0.5 + 0.5, 1.0 );
}

```



2.3. Robot csillogás

Feladat

A robot csillogási együtthatója (hatványkitevő a spekuláris árnyalási komponensben) a shine.png textúrából legyen betöltve a következő módon. Ha a vörös csatornából kiolvasott érték 0, állítsuk az együtthatót is 0-ra. Ha nem 0, akkor a kiolvasott értéket szorozzuk meg 16-tal és ez legyen a hatványkitevő.

Megoldás

Vegyünk fel egy harmadik textúra azonosítót a header-ben, mondjuk `m_shineTextureID` néven és töltsük ki az `InitTextures` és a `CleanTextures` eljárásokat ugyanúgy, mint korábban.

A robot kirajzolásakor adjuk át az új textúrát is:

```
void CMyApp::RenderRobot()
{
    //Anyagjellemzők beállítása
    glUniform3fv( ul("Ka"), 1, glm::value_ptr(glm::vec3(1, 1, 1)));
    glUniform3fv( ul("Kd"), 1, glm::value_ptr(glm::vec3(1, 1, 1)));
    glUniform3fv( ul("Ks"), 1, glm::value_ptr(glm::vec3(1, 1, 1)));

    //Textúra
    glBindTextureUnit(0, m_robotTextureID);
    glBindTextureUnit(1, m_shineTextureID);
    glBindSampler(0, m_SamplerID);
    glBindSampler(1, m_SamplerID);
    glUniform1i( ul("textureImage"), 0);
    glUniform1i( ul("textureShinning"), 1);

    glUniform1i( ul("state"), SHADER_STATE_ROBOT);

    ...
}
```

Vegyük fel az új textúrát a shader-ben is ugyan azzal a névvel, mint amivel átadtuk a **RenderRobot**-ban. Figyeljük meg, hogy magát a hatványkitevőt a **material.Shininess** reprezentálja. Kezdjük azzal, hogy a textúrából olvasott r érték 16-szorosára módosítjuk.

```
uniform sampler2D textureShinning;

...

void main()
{
    ...

    MaterialProperties material;
    material.Ka = Ka;
    material.Kd = Kd;
    material.Ks = Ks;
    material.Shininess = Shininess;

    if (state == SHADER_STATE_ROBOT)
    {
        float FragShininess = texture(textureShinning, textureCoords).r;
        if ( FragShininess > 1e-6 )
        {
            material.Shininess = FragShininess * Shininess;
        }
    }

    ...
}
```

Mivel a feladat azt kéri, hogy amennyiben az r komponens 0 (kisebb, mint ε), nem a kitevő, hanem az egész spekuláris komponens legyen 0 (és $-\infty$ -t nem tudunk átadni kitevőnek); módosítsuk a K_s adattagot.

```
if (state == SHADER_STATE_ROBOT)
{
    float FragShininess = texture(textureShinning, textureCoords).r;
    if ( FragShininess > 1e-6 )
    {
        material.Shininess = FragShininess * Shininess;
    }
    else
    {
        material.Ks = vec3(0.0);
    }
}
```

3. Interakció

3.1. Robot mozgatása

Feladat

A robot mozgatását oldjuk meg interaktívan az alábbiak szerint. A felhasználó Ctrl+kattintás segítségével tudjon kiválasztani egy pontot az asztallapon.

Debug: a kiválasztott pontba rajzolódjon ki egy kis kocka (a kocka megjelenítését lehessen ki-be kapcsolni egy GUI-s Checkbox-on, valamint a kocka méretét lehessen állítani egy GUI-s Slider segítségével [5,50] tartományban).

A kattintás hatására a robot egyenletes sebességgel forduljon (a saját függőleges tengelye körül) a pont felé, majd haladjon a kiválasztott pontba. [Részpontok: 0. azonnal a célhelyre teleporál; 1. nem fordul, csak odacsúszik; 2. azonnal odafordul utána lineárisan halad; 3. (teljes): odafordul és lineárisan halad]

A CalculateLegAnim függvénnyel való láb animáció csak akkor történjen, ha a robot mozog (azaz forog vagy halad). Amikor a robot megáll, álljon meg az animáció is abban az állapotban, ahol épp tartott, és ha a robot tovább indul, ugyaninnen folytatódjon az animáció.

A fordulás és a haladás sebessége is legyen állítható GUI-ról Slider segítségével. Ha mozgás közben történik az átállítás, az is helyesen legyen hatással a további mozgásra.

Megoldás

A feladatléírás alapján a robot mozgásának 3 állapota van: helyben áll/fordul/mozog. Ezen felül a robot rendelkezni fog cél pozícióval, valamint jelenlegi pozícióval. Bővítsük ki a **RobotState**-et ezekkel, valamint a két állítható sebességgel:

```
struct RobotState
{
    static const int ROBOT_STATE_STAND    = 0; //Nem mozog
    static const int ROBOT_STATE_ROTATING = 1; //forog
    static const int ROBOT_STATE_MOVING   = 2; //mozog

    int state = ROBOT_STATE_STAND;

    glm::vec3 position = glm::vec3(0, 0, 0);
    float rotation = 0;

    float movementSpeed = 20.0f;
    float rotationSpeed = 90.0f;

    glm::vec3 desiredPosition = glm::vec3(0, 0, 0);

    //A láb csak akkor mozog, ha a robot nem áll
    float legTime = 0.0f;
};
```

A két sebességet szabályozó csúszkát tegyük ki ImGui-ra.

A fenti adatok kiszámítására hozzunk létre külön eljárást, nevezzük **CalculateRobotPosition**-nek. Mivel a legTime-et most már feltételesen növeljük, vegyük át paraméterül az előző frame óta eltelt időt és emeljük át a legTime növelését az eljárásba. A helyén (az **Update**-ben) hívjuk meg ezt az eljárást a kapott **deltaTime** változó átadásával.

```
void CMyApp::CalculateRobotPosition(float deltaTime)
{
    if (m_robotState.state == RobotState::ROBOT_STATE_STAND) { return; }
    m_robotState.legTime += deltaTime;
}
```

Mielőtt a többi adatot ki tudnánk számítani, el kell végeznünk a sugármetszést a cél pozíció kiszámításához. A **MouseDown** eljárásban már elő van készítve az ehhez szükséges **m_PickedPixel**. Ezt felhasználva fogjuk az **Update**-ben elvégezni a sugármetszést. Itt szintén elő van készítve a metszéshez szükséges sugár. A mi feladatunk a sugárral elmetezni az asztallap síkját, majd megnézni, hogy ha van metszéspont, az az asztallapon belül van-e.

Hozzunk erre létre egy új függvényt, melyben elvégezzük a metszést. A függvény térjen vissza egy `bool` értékkel, ami azt jelzi, hogy a sugár metszi-e az asztallapot. Emellett amennyiben metszi, töltsük fel a referencia paraméterként kapott `vec3`-at a metszésponttal.

Bár az asztallap úgy van elhelyezve, hogy a metszés kivételesen triviálisan egyszerű, gyakorlásképp használjuk most is a `HitPlane` függvényt. E függvény paraméterei a sugár (`ray`), a sík egy pontja (`planeQ`), legyen most az asztallap egyik sarokpontja, a síkot kifeszítő két vektor (`planeI` és `planeJ`, nézzenek a két szomszédos sarokba), valamint a referenciaként átadott `Intersection` metszet, amit feltölt, amikor metszi a sugár a síkot. A `HitPlane` is visszaad egy `bool`-t, ami a metszéspont létezését jelenti. Az `Intersection`-be töltött adat `uv` tagja a sík koordináta-rendszerében lesz, az `x` és az `y` koordináták a `planeQ` origótól való, `planeI` és `planeJ` menti távolságok `planeI` és `planeJ` hosszaiban kifejezve. Ennek alapján, ha a két vektor hosszát az asztallap hosszának vesszük, pontosan akkor esik a metszéspont az asztallapon belülre, ha az `x` és `y` koordináták 0 és 1 közé esnek.

```
bool CMyApp::intersectTableTop(const Ray& ray, glm::vec3& result)
{
    //Sík
    glm::vec3 planeQ = TABLE_POS + 0.5f * glm::vec3(-TABLE_SIZE.x,
        TABLE_SIZE.y, -TABLE_SIZE.z);
    glm::vec3 planeI = glm::vec3(TABLE_SIZE.x, 0.0f, 0.0f);
    glm::vec3 planeJ = glm::vec3(0.0f, 0.0f, TABLE_SIZE.z);
    Intersection inter;
    //Ha van találkozás és az a négyzeten belül van
    if (HitPlane(ray, planeQ, planeI, planeJ, inter)
        && (inter.uv.x >= 0 && inter.uv.x <= 1)
        && (inter.uv.y >= 0 && inter.uv.y <= 1)
        ) {

        result = inter.position;
        return true;
    }
    else {
        return false;
    }
}
```

Vegyünk fel adattagot a metszéspontban kirajzolandó kocka méretének (`m_robotCubeGoalSize`), illetve annak, hogy ki kell-e rajzolni (`m_drawRobotGoal`), majd tegyük ki `ImGui`-ra a debug-jelölőnégyzetet és a kocka méretét szabályozó csúszkát.

Egészítsük ki a **Render**-t a kocka kirajzolásával:

```
void CMyApp::Render()
{
    ...

    RenderRobot();
    RenderTable();

    //Debug kocka, a robot célja
    if (m_drawRobotGoal) {
        glUniform1i( ul("state"), SHADER_STATE_DEFAULT);
        glm::mat4 goalWorldTr = glm::translate(m_robotState.desiredPosition) *
            glm::scale(glm::vec3(m_robotCubeGoalSize));
        DrawObject(m_cubeGPU, goalWorldTr);
    }

    ...
}
```

Hozzunk létre egy új eljárást (**CalculateRobotMovementData**), ami módosítja a robot mozgásának állapotát és cél pozícióját.

```
void CMyApp::CalculateRobotMovementData(glm::vec3 desiredPosition)
{
    m_robotState.desiredPosition = desiredPosition;
    m_robotState.state = RobotState::ROBOT_STATE_ROTATING;
}

void CMyApp::Update()
{
    ...

    if (m_IsPicking) {
        //Sugár indítása a kattintott pixelen át
        Ray ray = CalculatePixelRay(glm::vec2(m_PickedPixel.x, m_PickedPixel.y));
        //Metszés az asztallappal
        glm::vec3 desiredPos;
        if (intersectTableTop(ray, desiredPos)) {
            CalculateRobotMovementData(desiredPos);
        }

        m_IsPicking = false;
    }

    ...
}
```

Térjünk vissza a félbehagyott **CalculateRobotPosition** eljáráshoz.

Számítsuk ki a cél pozíció és a jelenlegi pozíció közötti különbséget (ez lesz a cél irány), majd forgassuk a robotot a megfelelő irányba az adott forgatási sebességgel:

```
void CMyApp::CalculateRobotPosition(float deltaTime)
{
    if (m_robotState.state == RobotState::ROBOT_STATE_STAND) { return; }

    m_robotState.legTime += deltaTime;

    // Az olvashatóságért használjunk referencia változókat a robot állapotának
    // gyakran használt részeihez
    glm::vec3& pos = m_robotState.position;
    float& rotation = m_robotState.rotation;

    glm::vec3 desiredDiff = m_robotState.desiredPosition - pos; // A kívánt
    // pozíció és a jelenlegi pozíció különbsége...
    float desiredDistance = glm::length( desiredDiff ); // ...és a távolságuk

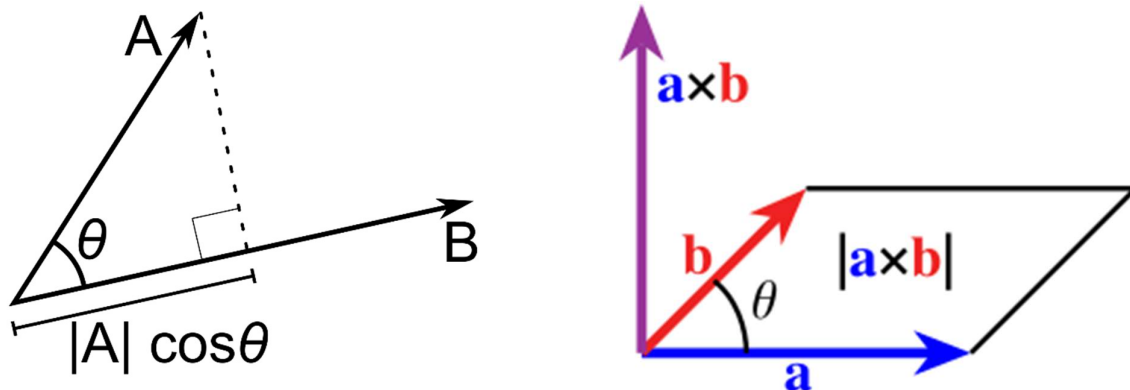
    //A robot előre néző iránya a jelenlegi forgás alapján
    glm::vec3 forward = glm::vec3( sinf( glm::radians( rotation ) ), 0.0f,
        cosf( glm::radians( rotation ) ) );

    switch ( m_robotState.state )
    {
        ***
    }
}
```

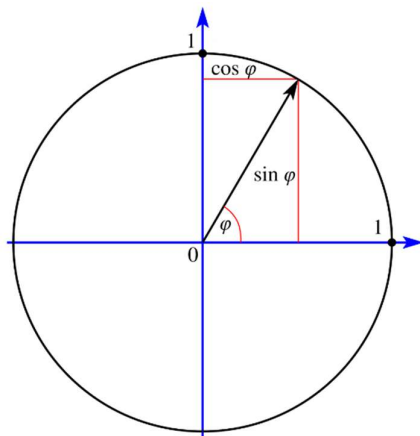
Emlékeztetőül:

Skaláris szorzat: $\langle \vec{a}; \vec{b} \rangle = \|\vec{a}\| \cdot \|\vec{b}\| \cdot \cos(\theta)$, ahol θ a két vektor által bezárt szög

Vektoriális szorzat: $\|\vec{a} \times \vec{b}\| = \|\vec{a}\| \cdot \|\vec{b}\| \cdot \sin(\theta)$, ahol θ a két vektor által bezárt szög. Mivel a mi esetünkben $\vec{a}$ és $\vec{b}$ az $y = 0$ síkba esnek, így a vektoriális szorzat eredményének, a síkra merőleges vektornak, egészen biztos, hogy csak az y komponense lesz nem nulla. Magyarul a hossza megegyezik az y komponens értékével.



A kapott sin és cos értékekből a szöget legkönnyebben az arctg-el nyerhetjük ki.



Ezek mellett $\vec{a}$ és $\vec{b}$ egységvektorok, így a hosszai a képlet jobb oldaláról elhagyhatók.

```
switch ( m_robotState.state )
{
case RobotState::ROBOT_STATE_ROTATING: // Forgatás
{
    glm::vec3 desiredDir = desiredDiff / desiredDistance; // A kívánt irányba
    // néző egységvektor

    // A kívánt irány és a jelenlegi előre néző irány közötti szög
    // kiszámítása a vektoriális és skaláris szorzat segítségével (atan2f)

    float desiredDeltaSinRotation = glm::cross( forward, desiredDir ).y;
    float desiredDeltaCosRotation = glm::dot( forward, desiredDir );

    // A kívánt forgás
    float desiredDeltaRotation = glm::degrees( ::atan2f( desiredDeltaSinRotation,
        desiredDeltaCosRotation ) );

    // A maximális forgás, amit megengedünk ebben a frame-ben
    float deltaRotation = m_robotState.rotationSpeed * deltaTime;

    if ( deltaRotation < fabsf( desiredDeltaRotation ) )
    {
        // Ha a kívánt forgás nagyobb, mint a megengedett maximális forgás,
        // akkor csak a megengedett maximális forgást hajtjuk végre...
        deltaRotation = copysignf( deltaRotation, desiredDeltaRotation );
        // ...De megőrizzük a forgás irányát (balra vagy jobbra)
        rotation += deltaRotation;
    }
    else
    {
        // Ha a kívánt forgás kisebb vagy egyenlő, mint a megengedett maximális
        // forgás, akkor a kívánt forgást hajtjuk végre,...
        rotation += desiredDeltaRotation;
        m_robotState.state = RobotState::ROBOT_STATE_MOVING;
        // ...És átállunk mozgás állapotba
    }
} break;
}
```

Amikor elértük a cél szöveget, a robot átválthat a haladás fázisba:

```
switch ( m_robotState.state )
{
    ...

case RobotState::ROBOT_STATE_MOVING:    //Mozgás
{
    // A kívánt irányba való mozgás vektora egy másodperc alatt a robot aktuális
    // sebességével
    glm::vec3 movingDiffPerSec = forward * m_robotState.movementSpeed;

    // A ténylegesen megtett mozgás vektora ebben a frame-ben
    glm::vec3 movingVector = movingDiffPerSec * deltaTime;

    pos += movingVector; // A robot pozíciójának frissítése a mozgás vektorral

    // Ha a kívánt pozíció közelebb van, mint a ténylegesen megtett mozgás
    // hossza, ...
    if ( desiredDistance <= glm::length( movingVector ) )
    {
        pos = m_robotState.desiredPosition; // ...akkor a robotot a kívánt
        // pozícióra helyezzük, ...
        m_robotState.state = RobotState::ROBOT_STATE_STAND; // ...és átállunk
        // álló állapotba
    }
}break;
}
```

Végezetül annyi maradt hátra, hogy eltranszformáljuk a robotot a jelenlegi pozíciójába és orientációjába. Mivel ez az összes darabját érinti a robotnak, célszerű a transzformálást a hierarchikus fa gyökérelemére, a testre alkalmazni:

```
void CMyApp::Update(const SUpdateInfo& updateInfo){

    ...

    m_robotTransform =
        glm::translate( m_robotState.position ) *
        glm::rotate( glm::radians( m_robotState.rotation ), glm::vec3( 0, 1, 0 ) );

    m_robotBaseTransform =
        m_robotTransform *
        glm::translate(TORSO_POS);

    ...

}
```

3.2. Robot lézer

Feladat

Imitáljunk egy robot által kilőtt lézerfényt a következők szerint.

- 1) Keressük meg, hogy az asztal melyik pontját nézi a robot (indítsunk sugarat a fejének (0, 5, 3.5) pontjából a (0, 0, 1) irányba, és metsszük az asztallap síkját)

- 2) Ha nem találja el a sugár az asztallapot, akkor ne változzon semmi
- 3) Ha eltalálja, akkor a metszéspont felett 1 egységgel hozzunk létre egy piros pontfényforrást (ez a korábbi irányfényforráson felül szintén hasson a színtérben az objektumokra).

A piros fény ne látszódjon az asztal lábain (csak a roboton és az asztallapon).

Megoldás

Vegyünk fel adattagokat egy második fényforráshoz a header-ben és a shader-ben is:

```
glm::vec4 m_lightPosition2 = glm::vec4(0.0f);

glm::vec3 m_La2 = glm::vec3(0.0, 0.0, 0.0);
glm::vec3 m_Ld2 = glm::vec3(1.0, 0.0, 0.0);
glm::vec3 m_Ls2 = glm::vec3(1.0, 0.0, 0.0);
```

```
uniform vec4 lightPosition2 = vec4( 0.0 );
uniform vec3 La2 = vec3(0.0, 0.0, 0.0 );
uniform vec3 Ld2 = vec3(1.0, 0.0, 0.0 );
uniform vec3 Ls2 = vec3(1.0, 0.0, 0.0 );
```

Végezzük el az **Update**-ben a sugármetszést. A sugár paramétereinek előállításához vegyünk fel egy függvényt, mondjuk **CalculateRobotHeadRay** névvel. A fejének pontját megkaptuk a ZHUtils.h header-ben, így nem kell begépelni. A sugár kezdőpontját úgy kapjuk ebből, hogy alkalmazzuk rá a robot fejének transzformációját. Ehhez természetesen elengedhetetlen, hogy előbb frissítsük a robot transzformációit. A robot kezdeti iránya a (0,0,1) vektor, az aktuálisat ennek alapján szintén a robot fejének transzformációjával kapjuk meg.

```
Ray CMyApp::CalculateRobotHeadRay() const
{
    Ray headRay;
    headRay.origin = glm::vec3( m_robotHeadTransform * glm::vec4(
        AVG_EYE_POS_OFFSET, 1 ) );
    headRay.direction = glm::vec3( m_robotHeadTransform *
        glm::vec4( 0, 0, 1, 0 ) );

    return headRay;
}
```

A korábban megírt `intersectTableTop` függvény most is alkalmazható. Ha van metszéspont, akkor toljuk el azt a (0,1,0) egységvektorral és csináljunk belőle pont fényforrást, ellenkező esetben vegyük fel a (0,0,0) irányú irány fényforrást.

```
void CMyApp::Update(const SUpdateInfo& updateInfo){  
  
    ...  
  
    //Robotból induló sugár metszése az asztallal  
    Ray headRay = CalculateRobotHeadRay();  
    glm::vec3 intersectionPoint;  
    if (intersectTableTop(headRay, intersectionPoint)) {  
        m_lightPosition2 = glm::vec4(intersectionPoint + glm::vec3(0, 1.0f, 0),  
                                     1.0f);  
    }  
    else {  
        m_lightPosition2 = glm::vec4(0.0f);  
    }  
}
```

Adjuk át a fényforrás adatait a `SetCommonUniforms` eljárásban:

```
void CMyApp::SetCommonUniforms()  
{  
  
    ...  
  
    if ( m_lightPosition2.w != 0.0f )  
    {  
        glUniform4fv( ul("lightPosition2"), 1,  
                      glm::value_ptr( m_lightPosition2 ) );  
        glUniform3fv( ul("La2"), 1, glm::value_ptr( m_La2 ) );  
        glUniform3fv( ul("Ld2"), 1, glm::value_ptr( m_Ld2 ) );  
        glUniform3fv( ul("Ls2"), 1, glm::value_ptr( m_Ls2 ) );  
    }  
    else  
    {  
        glUniform3fv( ul("La2"), 1, glm::value_ptr( glm::vec3( 0, 0, 0 ) ) );  
        glUniform3fv( ul("Ld2"), 1, glm::value_ptr( glm::vec3( 0, 0, 0 ) ) );  
        glUniform3fv( ul("Ls2"), 1, glm::value_ptr( glm::vec3( 0, 0, 0 ) ) );  
    }  
  
    ...  
}
```

Mindezek után a shader-ben, amikor az asztallapot vagy a robotot rajzoljuk, adjuk a másik fény értékhez ezt az új fény értéket.

```
void main()
{
    ***

    vec3 shadedColor = lighting(light, worldPosition, normal, material);

    if ( state == SHADER_STATE_TABLE || state == SHADER_STATE_ROBOT)
    {
        LightProperties light2 = light;
        light2.pos = lightPosition2;
        light2.La = La2;
        light2.Ld = Ld2;
        light2.Ls = Ls2;
        shadedColor += lighting(light2, worldPosition, normal, material);
    }

    ***
}
```



3.3. 3rd person camera

Feladat

A GUI-n legyen egy Checkbox, aminek az aktiválásakor az alábbi valósuljon meg. A kamera kerüljön fixen a robot feje mögé (és valamivel fölé), és nézzen a robottal azonos irányba, a kamerát ne lehessen mozgatni. Ha a robot mozog, a kamera kövesse. (Ez a feladat csak akkor ér pontot, ha a robot tud mozogni és elfordulni is; ha a Robot mozgatása feladat nincs

elkészítve, helyettesíthető GUI-s eltolás és forgatás paraméterekkel.) Ha a Checkbox nem aktív, álljon vissza a kamera régi működése.

Megoldás

Vegyünk fel egy bool adattagot, amiben a kamera módot tároljuk, mondjuk `m_isTopCamera` néven. Tegyük ki ImGui-ra a korábbi módon. Az **Update** eljárásban a kamera irányát már kiszámítottuk a robot pozíciójából és orientációjából. A kamera pozícióját is majdnem ugyanúgy kell kiszámítani, mindössze még el kell tolnunk mondjuk a (0, 60, -150) vektorral.

```
if ( m_isTopCamera )
{
    glm::vec3 camPosInObjSpace = glm::vec3( 0.0f, 60.0f, -150.0f );
    glm::mat4 matCam = glm::translate( m_robotState.position ) * glm::rotate(
        glm::radians( m_robotState.rotation ), glm::vec3( 0.0f, 1.0f, 0.0f ) );
    glm::vec3 camPos = glm::vec3( matCam * glm::vec4(camPosInObjSpace,1.0f ) );

    m_camera.SetView(
        camPos, //Honnan nézzük a színteret
        m_robotState.position, //A színtér melyik pontját nézzük
        glm::vec3( 0.0f, 1.0f, 0.0f ) ); //Felfelé mutató irány a világban
}
else
{
    m_cameraManipulator.Update(updateInfo.DeltaTimeInSec);
}
```

3.4. Debug shader rendering

Feladat

GUI-n lenyíló menü (ImGui::Combo) segítségével válthassunk a következő debug nézetek között. Az alapbeállítás a korábbiaknak megfelelő színezés (árnyalás és textúra). További beállítások: 1) normálvektor (mint szín) megjelenítése, 2) csak az árnyalás (textúra nélkül), 3) csak a diffúz textúra, 4) csak a shininess textúra.

Megoldás

Vegyünk fel 5 konstans adattagot a különböző debug módoknak egy adattagot, mondjuk `m_shaderDebugState` néven, 0 kezdeti értékkel. Tegyük ki ImGui-ra:

```
const int SHADER_DEBUG_NONE = 0;
const int SHADER_DEBUG_NORMALS = 1;
const int SHADER_DEBUG_SHADING = 2;
const int SHADER_DEBUG_DIFFUSE_TEXTURE = 3;
const int SHADER_DEBUG_SHINE_TEXTURE = 4;

int m_shaderDebugState = SHADER_DEBUG_NONE;
```

```
ImGui::SeparatorText("Debug mód");
static const std::array<const char *, 5> debugModes = {
    "Kikapcsolva (árnyalás és textúra)",
    "Normálvektor",
    "Csak az árnyalás (textúra nélkül)",
    "Csak diffúz textúra",
    "Csak a shininess textúra"
};
ImGui::Combo("Aktív mód", &m_shaderDebugState, debugModes.data(),
    debugModes.size());
```

Adjuk át az adattagot uniform-ként!

```
const int SHADER_DEBUG_NONE = 0;
const int SHADER_DEBUG_NORMALS = 1;
const int SHADER_DEBUG_SHADING = 2;
const int SHADER_DEBUG_DIFFUSE_TEXTURE = 3;
const int SHADER_DEBUG_SHINE_TEXTURE = 4;

uniform int debugState = SHADER_DEBUG_NONE;
```

A shader-ben a `main` eljárás végén írjuk felül a szint a kiválasztott módnak megfelelően:

```
void main()
{
    ...

    if ( debugState == SHADER_DEBUG_NORMALS )
    {
        outputColor = vec4( normalize( worldNormal ) * 0.5 + 0.5, 1.0 );
    }
    if ( debugState == SHADER_DEBUG_SHADING )
    {
        outputColor = vec4( shadedColor, 1.0 );
    }
    if ( debugState == SHADER_DEBUG_DIFFUSE_TEXTURE )
    {
        outputColor = texture(textureImage, textureCoords);
    }
    if ( debugState == SHADER_DEBUG_SHINE_TEXTURE )
    {
        outputColor = texture(textureShinning, textureCoords);
    }
}
```

